The Server-side JavaScript

Day 1

Eng. Hany Saad
SD Dept.
ITI – Assiut Branch

Introduction: Basic

- In simple words Node.js is **'server-side JavaScript platform'**.
- Node.js is a **platform for JavaScript applications running out side browser.**
- Node.js is a high-performance **network applications framework.**
- Its an **Open Source, Cross-platform, runtime environment for server-side and a high performance networking applications.**
  - **Cross platform:** can be installed on UNIX/Linux/Mac OS X, SunOS and Windows.
- Node is the official name of the project, NodeJS.
- Created by Ryan Dahl in 2009.
- Its built in v8 chrome engine (**Node.js** runs on V8).
  - **V8** is an open source JavaScript engine developed by Google. Its written in C++ and is used in Google Chrome Browser.
- **Very Fast** Being built on Google Chrome's V8 JavaScript Engine, Node.js library is very fast in code execution.
- It's a **command line** tool.
- From the official site:

  'Node's goal is to provide an easy way to build scalable network programs' - (from nodejs.org!)

Introduction: Advanced

- Node.js uses an **event-driven, non-blocking I/O** model, which makes it lightweight. (from nodejs.org!)
- It makes use of **event-loops** via JavaScript's **callback** functionality to implement the non-blocking I/O.
- Programs for Node.js are written in JavaScript but not in the same JavaScript we are use to. There is no DOM implementation provided by Node.js, i.e. you **can not** do this:

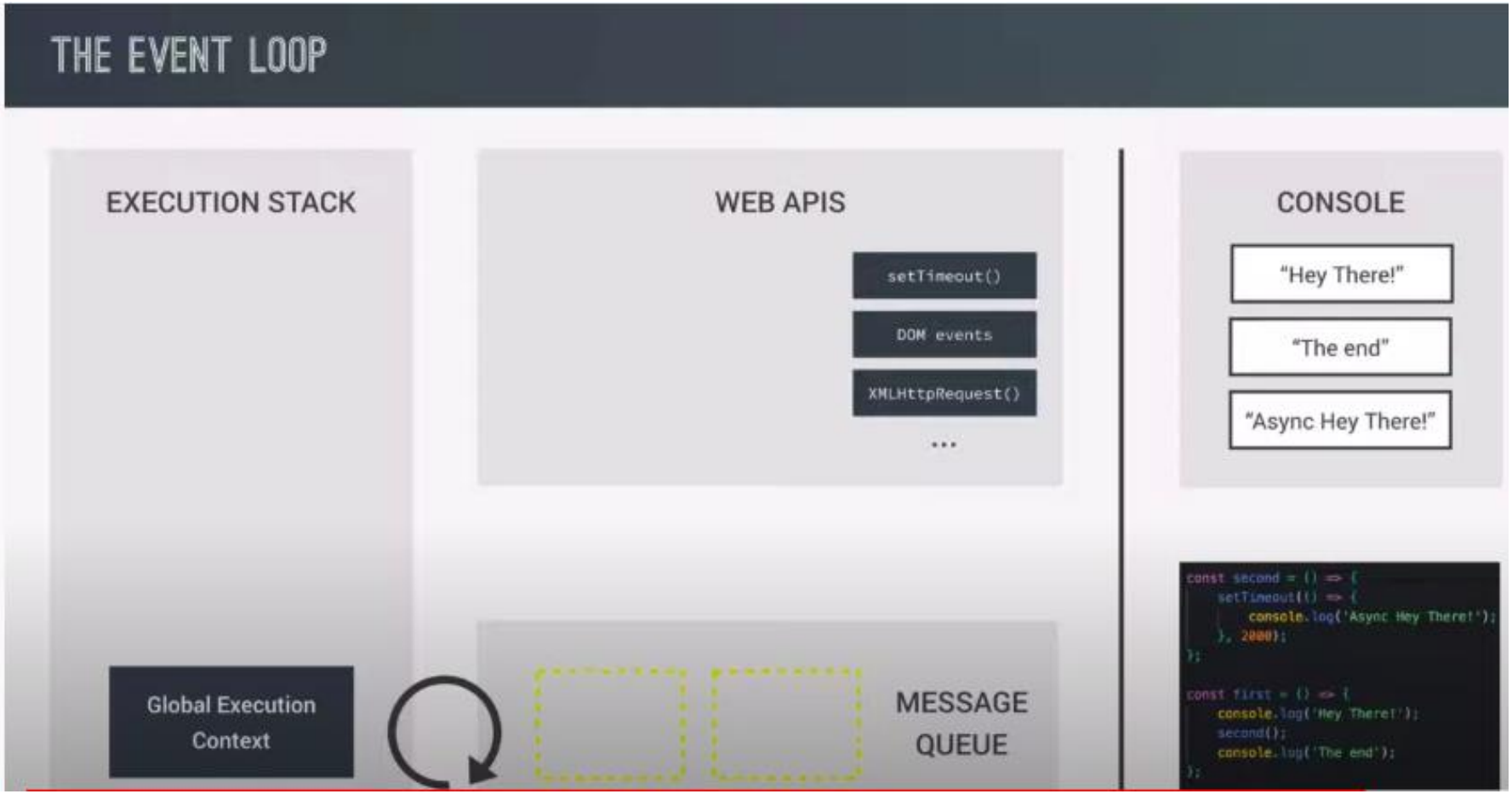
```
var element = document.getElementById("elementId");
```

- Node.JS is a **single-thread (Single Threaded but Highly Scalable)**.
- **No Buffering** - Node.js applications never buffer any data. These applications simply output the data in chunks.
- In 'Node.js' , '**.js**' doesn't mean that its solely written JavaScript. It is 40% JS and 60% C++.

Some Theory: Event Driven

- A code block gets into the queue of execution when an event happens
  - It gets executed on its turn
- Anyone can raise (or listen to) an event
  - System
  - Some library (module)
  - Your code
- You have the choice (and ways) to attach a code block to an event
  - Also known as event callbacks

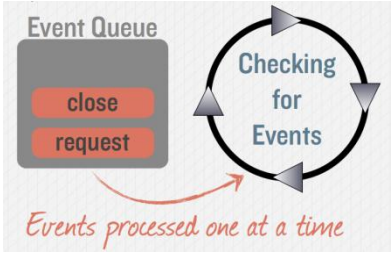
Some Theory: Event-loops

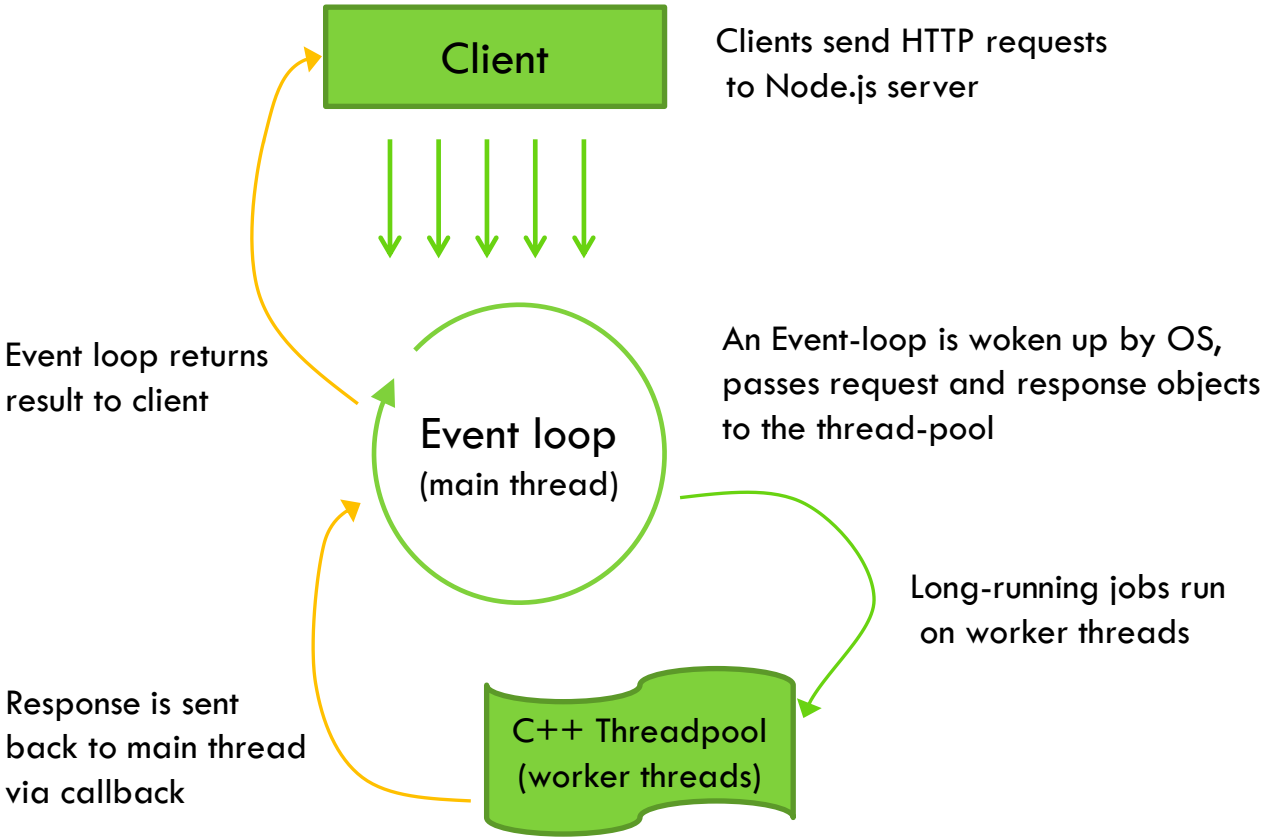


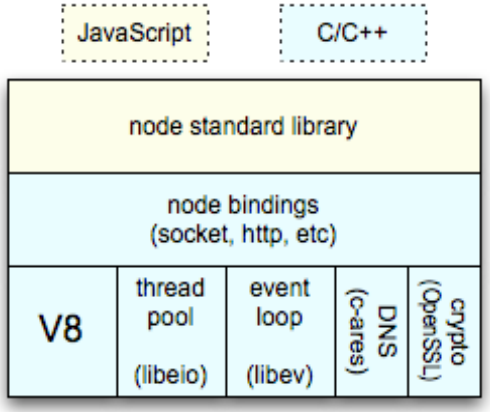
https://www.youtube.com/watch?v=F_45gS6FODo

Some Theory: Event-loops (Cont.)

- Event-loops are the core of event-driven programming, almost all the UI programs use event-loops to track the user event, for example: Clicks, Ajax Requests etc.
- In an event-driven application, there is generally a main loop that listens for events, and then triggers a callback function when one of those events is detected.







Some Theory: Event Callback

- Because the system is single threaded
  - And we do not want to block it for I/O
  - We use asynchronous functions for getting work done
- We depend on the events to tell when some work is finished
  - And we want some code to execute when the event occurs
- Asynchronous functions take a function as an additional argument and call the function when the event occurs
  - This function is known as callback function
- Example:

```
db.query('SELECT * from huge_table', function(res) {
      console.log('result a:', res);
});
```

Some Theory: Non-Blocking I/O

- Traditional I/O

```
var result = db.query("select x from table_Y");
doSomethingWith(result); //wait for result!
doSomethingWithOutResult(); //execution is blocked!
```

- Non-traditional, Non-blocking I/O

```
db.query("select x from table_Y",function (result){
  doSomethingWith(result); //wait for result!
});
doSomethingWithOutResult(); //executes without any delay!
```

Some Theory: Single Threaded

- Only one code executes at a time
  - Everything else remains in the queue
  - No worry about different portions of code accessing the same data structures at the same time
- Delay at one place delays everything after that
- Can not take advantage of multi-core CPU

Some Theory: Asynchronous

- Not all code is executed in the same order it is written
- Node (and you) divide the code into small pieces and fire them in parallel
- Typically the code announces its state of execution (or completion) via events

What can you do with Node.js ?

- Node.js is designed for **DIRT applications.**
  - DIRT stands for : **data-intensive real-time applications**
  - e.g. video streaming, SPAs, networking apps
- Following are the areas where Node.js is perfect technology:
  - I/O bound Applications
  - Data Streaming Applications
  - Data Intensive Real time Applications (DIRT)
  - JSON APIs based Applications
  - Single Page Applications
- You can create an **HTTP server** and print '*hello world*' on the browser in just 4 lines of JavaScript.
- You can create a **TCP server** similar to HTTP server, in just 4 lines of JavaScript.
- You can create a **DNS server.**
- You can create a **Static File Server.**
- You can create a **Web Chat Application** like GTalk in the browser.
- Node.js can also be used for creating online games, collaboration tools or **anything which sends updates to the user in real-time.**

When to use Node.js?

- Node.js is good for creating **streaming based real-time services, web chat applications, static file servers** etc.
- If you need high level concurrency and not worried about CPU-cycles.
- If you are great at writing JavaScript code because then you can use the same language at both the places: server-side and client-side.
- More can be found at: http://stackoverflow.com/questions/5062614/how-to-decide-when-to-use-nodejs

When to not use Node.js?

- When you are doing heavy and CPU intensive calculations on server side, because event-loops are CPU hungry.
- Node.js is a no match for enterprise level application frameworks like Spring(java), Django(python), Symfony(php), **ASP.Net SignalR (Microsoft)** … etc. Applications written on such platforms are meant to be highly user interactive and involve complex business logic.
- Node can't run on GUI, but run on terminal/cmd
- Read further on disadvantages of Node.js on Quora: http://www.quora.com/What-are-the-disadvantages-of-using-Node-js

Who is using Node.js in production?

- **Yahoo!** : iPad App **Livestand** uses Yahoo! Manhattan framework which is based on Node.js.
- **LinkedIn** : LinkedIn uses a combination of Node.js and MongoDB for its mobile platform. iOS and Android apps are based on it.
- **eBay** : Uses Node.js along with ql.io to help application developers in improving eBay's end user experience.
- **Dow Jones** : The WSJ Social front-end is written completely in Node.js, using Express.js, and many other modules.
- **Microsoft, PayPal, Uber,** …
- Complete list can be found at: https://github.com/joyent/node/wiki/Projects,-Applications,-and-Companies-Using-Node

Environment Setup

- To set up your environment for Node.js, you need the following two software available on your computer:
  - **Text Editor**
    - Will be used to type your program
    - You can Use any text editor (For example: Notepad, VS code,…etc).
    - The source files for Node.js programs are typically named with the extension **".js"**.
  - **The Node.js binary installable.**
    - The source code written in source file is simply javascript. The Node.js interpreter will be used to interpret and execute your javascript code.
    - Node.js distribution comes as a binary installable for SunOS , Linux, Mac OS X, and Windows operating systems with the 32-bit (386) and 64-bit (amd64) x86 processor architectures.
    - You can download it from here: https://nodejs.org/en/download
    - After downloading it, you to install it.

Example-1: Getting Started & Hello World

- Open your favorite editor and start typing JavaScript.
- Here is a simple example, which prints '*hello world*'
  - Ex.1:

```
/* Hello, World! program in node.js */
console.log("Hello, World!");
```

  - Ex.2:

```
/* Hello, World! program in node.js */
console.log("Hello, ");

setTimeout(function(){
    console.log("NodeJS world!");
}, 3000);
//it prints 'hello' first and waits for 3 seconds and then prints
  'world'
```

- When you are done, open cmd/terminal and type this:

```
node YOUR_FILE.js
```

Node.js Ecosystem

- Node.js heavily relies on **modules**.
- Creating a module is easy, just put your JavaScript code in a separate js file and include it in your code by using keyword require, like:

```
var modulex = require('./modulex');
```

- Module is a self contained series of one or more .js files presented by an object.
- Modules is where we can encapsulate related functionality into a single file.
- Should be accessible from outside the file.
- Modules act as libaries
  - It's a set reusable code that adds extra functionality to our application.
- Modules allow Node to be extended.
  - It allows us to install packages/modules, and publish our custom one and share them with other developers.
- Node provides core modules that can be included by their name:
  - File System – require('fs')
  - Http – require('http')
  - Utilities – require('util')

Node.js - First Application (HTTP Module)

- A Node.js application consists of following three important parts:
  - **Import required modules** – We use **require** directive to load a Node.js module.
  - **Create server** – A server which will listen to client's request similar to Apache HTTP Server.
  - **Read request and return response** – server created in earlier step will read HTTP request made by client which can be a browser or console and return the response.

Node.js - First Application (HTTP Module) – Cont.

- **Creating Node.js Application:**
  - **Step 1: Import required module**
    - We use **require** directive to load http module and store returned HTTP instance into http variable as follows

```
var http = require("http");
```

  - **Step 2: Create Server**
    - At next step we use created http instance and call **http.createServer()** method to create server instance and then we bind it at port 8081 using **listen** method.

```
http.createServer(function (request, response) {

   // Send the HTTP header
   // HTTP Status: 200 : OK
   // Content Type: text/plain
   response.writeHead(200, {'Content-Type': 'text/plain'});

   // Send the response body as "Hello World"
   response.end('Hello World\n');
}).listen(8081);

// Console will print the message
console.log('Server running at http://127.0.0.1:8081/');
```

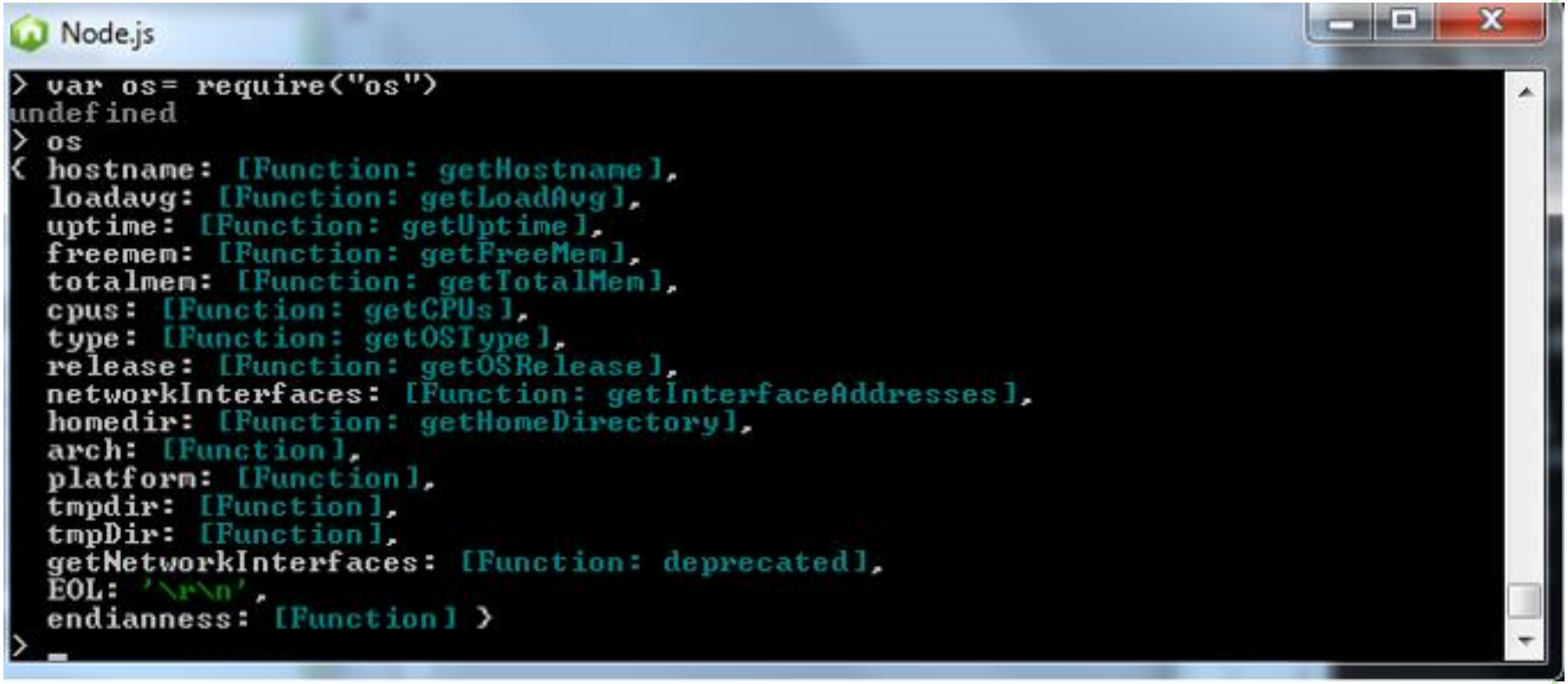
Node.js - First Application (HTTP Module) – Cont.

- **Creating Node.js Application (Cont.):**
  - **Step 3: Testing Request & Response**
    - Let's put step 1 and 2 together in a file called **main.js.**
    - Open command prompt and execute the main.js to start the server

      node main.js
    - Make a request to Node.js server
      - Open http://127.0.0.1:8081/ in any browser and see the below result.

OS Module

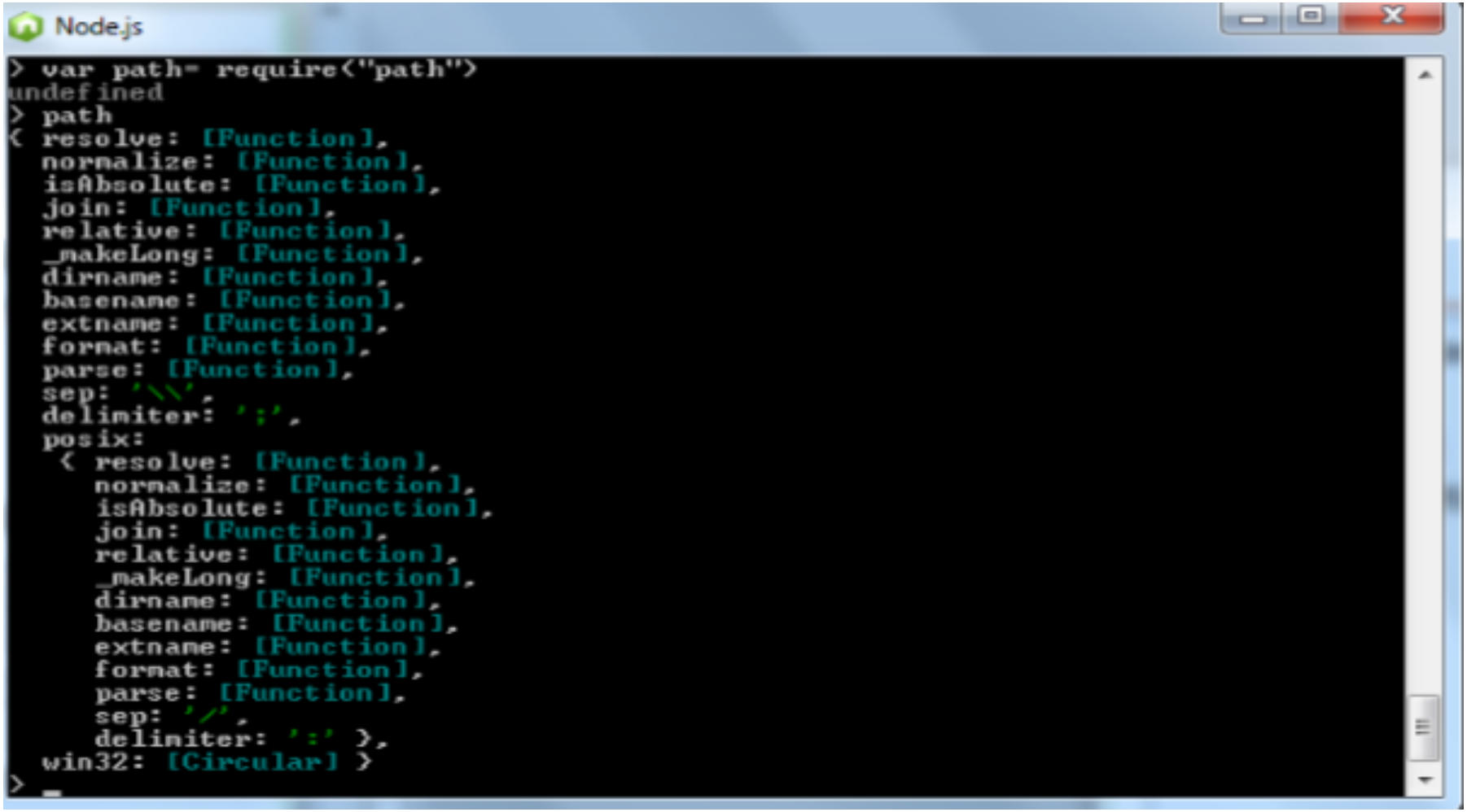
- Module for info about operating system of application
- Useful for statistics



- More details and Examples:
  - http://www.tutorialspoint.com/nodejs/nodejs_os_module.htm

PATH MODULE

- module contains utilities for handling and transforming file paths



- More Details and Examples:
  - http://www.tutorialspoint.com/nodejs/nodejs_path_module.htm

FILE SYSTEM MODULE

- For reading and writing to files.
- All its methods have asynchronous and synchronous forms.

```
//file is already created
//this happens Asynch-->non blocking
var fs=require("fs");
console.log("starting");

fs.readFile("readingFile.txt",function(err,data){
    console.log("this is a new way to read");
    console.log("content: "+data)

});
console.log("exec");
```

```
// to make it synch -->blocking
var fs=require("fs");
console.log("starting");

var data=fs.readFileSync("readingFile.txt");
console.log("this is a new way to read");
console.log("content: "+data);
console.log("exec");
```

- More Details and Examples: http://www.tutorialspoint.com/nodejs/nodejs_file_system.htm
  - NodeJS Buffers: http://www.tutorialspoint.com/nodejs/nodejs_buffers.htm

Node.js - Streams

- Streams are objects that let you read data from a source or write data to a destination in continous fashion.
- In Node.js, there are four types of streams.
  - **Readable** - Stream which is used for read operation.
  - **Writable** - Stream which is used for write operation.
  - **Duplex** - Stream which can be used for both read and write operation.
  - **Transform** - A type of duplex stream where the output is computed based on input.
- Each type of Stream is an **EventEmitter** instance and throws several events at different instance of times. For example, some of the commonly used events are:
  - **data** - This event is fired when there is data is available to read.
  - **end** - This event is fired when there is no more data to read.
  - **error** - This event is fired when there is any error receiving or writing data.
  - **finish** - This event is fired when all data has been flushed to underlying system

Node.js – Streams (Read From Stream)

- Reading from Stream:

```
var fs = require("fs");
var data = '';

// Create a readable stream
var readerStream = fs.createReadStream('input.txt');

// Set the encoding to be utf8.
readerStream.setEncoding('UTF8');

// Handle stream events --> data, end, and error
readerStream.on('data', function(chunk) {
   data += chunk;
});

readerStream.on('end',function(){
   console.log(data);
});

readerStream.on('error', function(err){
   console.log(err.stack);
});

console.log("Program Ended");
```

Node.js – Streams (Write to Stream)

- Writing to Stream:

```
var fs = require("fs");
var data = 'Simply Easy Learning';

// Create a writable stream
var writerStream = fs.createWriteStream('output.txt');

// Write the data to stream with encoding to be utf8
writerStream.write(data,'UTF8');

// Mark the end of file
writerStream.end();

// Handle stream events --> finish, and error
writerStream.on('finish', function() {
    console.log("Write completed.");
});

writerStream.on('error', function(err){
   console.log(err.stack);
});

console.log("Program Ended");
```

Node.js – Streams (Piping Streams)

- Piping streams:
  - Piping is a mechanism where we provide output of one stream as the input to another stream.
  - It is normally used to get data from one stream and to pass output of that stream to another stream.
  - There is no limit on piping operations.
  - Example:

```
var fs = require("fs");

// Create a readable stream
var readerStream = fs.createReadStream('input.txt');

// Create a writable stream
var writerStream = fs.createWriteStream('output.txt');

// Pipe the read and write operations
// read input.txt and write data to output.txt
readerStream.pipe(writerStream);

console.log("Program Ended");
```

Node.js - npm

- **npm** (Node Package Manager) comes bundled with Node.js installation.
  - It is a package manager for Node.js
  - It's a "CLI" command line interface.
- Node Package Manager (npm) provides following two main functionalities:
  - Online repositories for node.js packages/modules which are searchable on search.nodejs.org
  - Command line utility to install Node.js packages, do version management and dependency management of Node.js packages.
- Installing Modules using npm
  - Use Command: npm install <Module Name>
  - After installing the module you can use it normally, using: require('Module name')

Node.js – npm (Global vs Local installation)

- **Local installation:**
  - To install module locally, use Command: npm install <Module Name>
  - By default, npm installs any dependency in the local mode.
  - Here local mode refers to the package installation in node_modules directory lying in the folder where Node application is present.
  - Locally deployed packages are accessible via require() method.
  - you can use **npm ls** command to list down all the locally installed modules.
- **Global Installation:**
  - To install module globally, use Command: npm install <Module Name> -g
  - Globally installed packages/dependencies are stored in system directory. Such dependencies can be used in CLI (Command Line Interface) function of any node.js but can not be imported using require() in Node application directly.
  - you can use **npm ls -g** command to list down all the locally installed modules.
- More about npm and modules: http://www.tutorialspoint.com/nodejs/nodejs_npm.htm

PACKAGE.JSON

- It's a file that contains txt info about project's loaded modules
- It can build or rebuild node-module folder
- Most important fields are name & version.

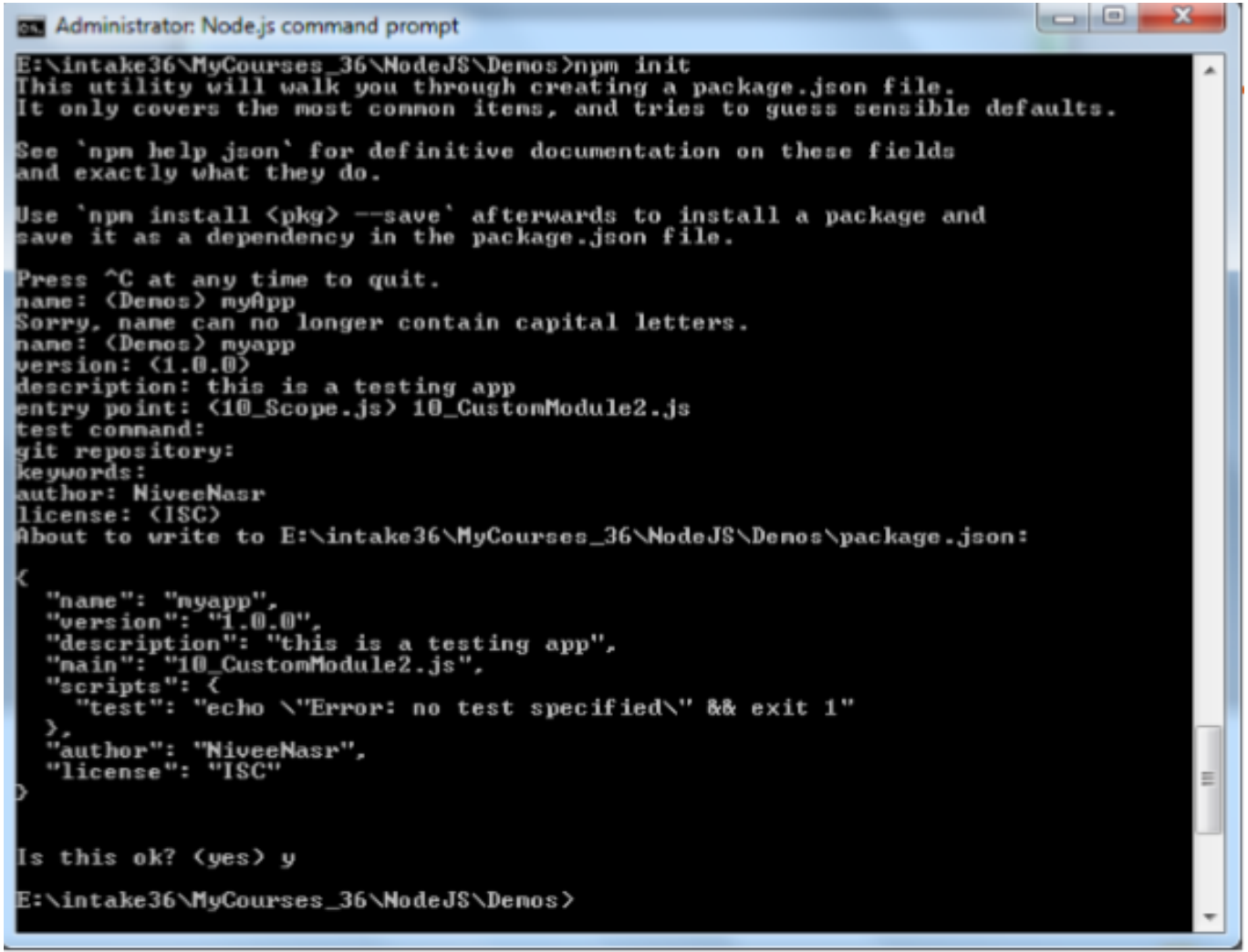
package.json

```
{
  "name": "myapp",
  "version": "1.0.0",
  "description": "this is a testing app",
  "main": "10_CustomModule2.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "NiveeNasr",
  "license": "ISC"
}
```

PACKAGE.JSON - INITIALIZATION

- Initialize it use:

  npm init



PACKAGE.JSON – NPM COMMANDS

- Initialize it use:
  - **npm init** - initialize a package.json file
  - **npm install** - install packages listed in package.json
  - **npm install pkg_nm -- save** - install packages and add it to dependencies in package.json file
  - **npm install pkg_nm --save -dev** - install packages and add it to devdependencies in package.json file
  - **npm uninstall pkg_nm --save -dev** - uninstall packages

Most Known NodeJs-based framewrks and modules

- Other frameworks that will make it easier using node
  - **Express** – to make things simpler e.g. syntax, DB connections.
  - **Jade, Ejs** – HTML template system (view engines)
  - **Socket.IO** – to create real-time apps
  - **Nodemon** – to monitor Node.js and push change automatically
  - **Redis** – in memory DB
  - **MongoDB**

Node.js - REPL Terminal

- REPL stands for **Read Eval Print Loop** and it represents a computer environment like a window console or Unix/Linux shell where a command is entered and system responds with an output in interactive mode.
- Node.js or Node comes bundled with a REPL environment. It performs the following desired tasks:
  - **Read** - Reads user's input, parse the input into JavaScript data-structure and stores in memory.
  - **Eval** - Takes and evaluates the data structure
  - **Print** - Prints the result
  - **Loop** - Loops the above command until user press **ctrl-c** twice.
- REPL feature of Node is very useful in experimenting with Node.js codes and to debug JavaScript codes.
- REPL is a quick way to write and execute js without creating a file

Node.js - REPL Terminal (Cont.)

- Online REPL Terminal:
  - http://www.tutorialspoint.com/nodejs_terminal_online.php
- Starting REPL:
  - REPL can be started by simply running node on shell/console without any argument as follows: node
  - Let's try simple mathematics at Node.js REPL command prompt:

```
> 1 + 3
4
> 1 + ( 2 * 3 ) - 4
3
>
```

Node.js - REPL Terminal (Cont.)

- Starting REPL (Cont.):
  - Using Variables:
    - You can make use variables to store values and print later like any conventional script.
    - If **var** keyword is not used then value is stored in the variable and printed.
    - Whereas if var keyword is used then value is stored but not printed.
    - You can print variables usind console.log().

```
> x = 10
10
> var y = 10
undefined
> x + y
20
> console.log("Hello World")
Hello World
undefined
```

Node.js - REPL Terminal (REPL Commands)

- REPL Commands:
  - **ctrl + c** - terminate the current command.
  - **ctrl + c twice** - terminate the Node REPL.
  - **ctrl + d** - terminate the Node REPL.
  - **Up/Down Keys** - see command history and modify previous commands.
  - **tab Keys** - list of current commands.
  - **.help** - list of all commands.
  - **.break** - exit from multiline expression.
  - **.clear** - exit from multiline expression
  - **.save *filename*** - save current Node REPL session to a file.
  - **.load *filename*** - load file content in current Node REPL session.

- More examples: http://www.tutorialspoint.com/nodejs/nodejs_repl_terminal.htm

Node.js - Debugging

- Node Inspect (Built-in):
  - Run inspector: node inspect myscript.js
  - How to use: https://nodejs.org/api/debugger.html
- Node-inspector (Package)
  - Installation: npm i node-inspector
  - How To use:
    - https://www.npmjs.com/package/node-inspector
    - https://docs.strongloop.com/display/SLC/Debugging+applications

Appendix-1: Resource to get started

- Read this tutorial: http://www.tutorialspoint.com/nodejs/index.htm
- Watch this video at Youtube: http://www.youtube.com/watch?v=jo_B4LTHi3I
- Read the free O'reilly Book '*Up and Running with Node.js*' @ http://ofps.oreilly.com/titles/9781449398583/
- Visit www.nodejs.org for Info/News about Node.js
- Watch Node.js tutorials @ http://nodetuts.com/
- For Info on MongoDB: http://www.mongodb.org/display/DOCS/Home
- For anything else **Google!**

Appendix-2: Some Good Modules

- **Express** – to make things simpler e.g. syntax, DB connections.
- **Jade** – HTML template system
- **Socket.IO** – to create real-time apps
- **Nodemon** – to monitor Node.js and push change automatically
- **CoffeeScript** – for easier JavaScript development
- Find out more about some widely used Node.js modules at: http://blog.nodejitsu.com/top-node-module-creators

The Server-side JavaScript

Thanks…

Eng. Hany Saad
SD Dept.
ITI – Assiut Branch